

Sports Data Decision Tree Report

1. Problem statement

The dataset is Sports Data. Each row is one Game, with average team age, Match light type, Match format, Bowlers in team, Wicket keeper in team, All rounder in team, First selection, Opponent, Season, Audience number, Offshore, Max run scored 1over, Max wicket taken 1over, Extra bowls bowled, Min run given 1over, Min run scored 1over, Max run given 1over, extra bowls opponent, player highest run, Players scored zero, player highest wicket. The Result column records whether that match resulted in winning or not.

So the problem is: using the sports detail available, predict whether the match will result in winning or not. If the sports can predict this, it can predict future games result as well.

Out of 2930, only 473 were won. That is 16.16% percent. The remaining did not win, so the data is heavily imbalanced. This matters later when we look at accuracy.

2. Target variable and independent variables

Target variable (the one we predict): Target, which takes only two values.

- 1 = Match won (473)
- 0 = Match lost (2457)

Independent variables (the inputs used to predict it): everything else that is left after dropping the columns explained in point 4.

There are no missing values in any column, so no imputation was needed.

3. Type 1 and Type 2 error

In this problem the two errors:

Type 1 error (false positive): the model says the match will result in win but it does not.

Type 2 error (false negative): the model says the match will result is losing, but it is won instead.

For future predictions the Type 1 error is the costlier one, because a lost match is lost revenue. The model here makes far more Type 1 errors than Type 2 errors.

4. Columns dropped and why

Game number is dropped as it is a unique code.

Season / Audience number / Audience number / Offshore / First_selection

is dropped as it can vary place to place and does not affect match's result.

5. Encoding used in the code

Scikit-learn cannot take text columns, so Match light type, Match format and Opponent had to be converted to numbers. Two different methods were used.

Gender, label encoding: Match light type

Occupation, one hot encoding: Match format and Opponent

Feature importance from the model. The tree measures how much each variable reduced the Gini impurity across all its splits.

Extra_bowls_bowled	0.127917
player_highest_run	0.125789
Max_run_scored_1over	0.101353
Avg_team_Age	0.074399
extra_bowls_opponent	0.073833
Max_wicket_taken_1over	0.067990
Min_run_scored_1over	0.065653
Min_run_given_1over	0.053844
Players_scored_zero	0.049881
All_rounder_in_team	0.048477
Bowlers_in_team	0.036598
player_highest_wicket	0.030289
Max_run_given_1over	0.025276
Opponent_Srilanka	0.017189
Match_light_type_Day and Night	0.016679
Opponent_England	0.014444
Opponent_Kenya	0.014215
Opponent_Australia	0.011816
Opponent_South Africa	0.011595
Opponent_Zimbabwe	0.010146
Match_light_type_Day	0.008044
Opponent_Pakistan	0.005605
Opponent_Bangladesh	0.005172
Match_format_T20	0.003103
Match_format_ODI	0.000693
Wicket_keeper_in_team	0.000000
Match_format_Test	0.000000
Match_format_20-20	0.000000
Match_light_type_Night	0.000000

Opponent_West Indies 0.000000

dtype: float64

Max run scored carry the most weight.

6. Train and test split, accuracy and confusion matrix

The data was split 70:30 with random state=1.

The first model was built with criterion='gini', min samples split=2 and min samples leaf=1, which are the default settings. This lets the tree grow with no limit at all.

Confusion matrix, training data (no pruning)

```
[80]: print(metrics.confusion_matrix(Y_train,Y_train_predict_class))  
  
[[1722   0]  
 [   0  329]]
```

Confusion matrix, test data (no pruning)

```
print(metrics.confusion_matrix(Y_test,Y_test_predict_class),'\n')  
tn, fp, fn, tp = metrics.confusion_matrix(Y_test,Y_test_predict_class).ravel()  
print('True Negative:',tn,'\n''False Positives:',fp,'\n''False Negatives:',fn,'\n''True Positives:',tp)  
  
[[687  48]  
 [ 26 118]]  
  
True Negative: 687  
False Positives: 48  
False Negatives: 26  
True Positives: 118
```

The training matrix has zeros in both error boxes. Not a single mistake. That is not a good model, that is a model that has memorised the training data. On the test data it drops to 92 percent. A 8% point fall from train to test is the clear sign of overfitting, which is why the tree was pruned.

7. Results before and after pruning

For pruning, three limits were set on the tree: max depth=13, min samples leaf=30 and min samples split=10. So the tree cannot go deeper than 13 levels, every leaf must hold at least 30 games, and a node needs at least 10 games before it is allowed to split.

Pruned model, confusion matrix on training data

```
[[1712  10]  
 [ 295  34]]
```

Pruned model, confusion matrix on test data

```
[[731   4]  
 [134  10]]
```

8. Is the model underfitting or overfitting?

The first model was clearly overfitting. The second model is not overfitting. Train and test are 85% and 84% percent, less than one point apart, so whatever the model learnt from the training data holds up on new data. On the train versus test test, the pruned model is a well fitted model.

Final Conclusion

The CART model was developed to predict game's result.

The preliminary decision tree was checked for overfitting by comparing training and testing accuracy. A large difference between the two indicates that the tree is learning the training data too closely.

To control overfitting, pruning parameters such as max depth, min samples split, and min samples leaf were introduced. These parameters reduce unnecessary branches and prevent the tree from creating very small, highly specific groups.

Therefore, the preferred final model is the pruned tree that provides better test performance with controlled complexity and an acceptable balance between Type I and Type II errors.